Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Arquitetura de Computadores III

Trabalho Prático 2: Pipeline RISC-V

Integrantes:

Andriel Mark da Silva Pinto
Beatriz Miranda Paiva
Felipe Henrique Oliveira Diniz
Matheus de Almeida Moreira
Otoniel Goulart Damasceno Filho

Belo Horizonte

2026

Conteúdo

1 Introdução **2**

2 Descrição do Desenvolvimento **2**
2.1 Arquitetura Implementada 2
2.2 Decisões de Projeto 3

3 Resultados Obtidos **3**
3.1 Parte 1 Branch e Controle de Fluxo 3
3.1.1 Descrição 3
3.1.2 Análise de Desempenho Parte 1 3
3.2 Parte 2 Execução com NOPs Estáticos 4
3.2.1 Descrição 4
3.2.2 Análise de Desempenho Parte 2 4
3.3 Parte 3 Hazard Detection e Forwarding em Hardware 5
3.3.1 Descrição 5
3.3.2 Análise de Desempenho Parte 3 5
3.4 Comparativo de Desempenho entre as Etapas 6
3.5 Validação Funcional 6

4 Conclusão **6**

5 Uso de Inteligência Artificial (IA) **7**

6 Repositório no GitHub **11**

1 Introdução

O processamento moderno de instruções em arquiteturas RISC (Reduced Instruction Set Computer) é largamente beneficiado pela técnica de *pipeline*, que permite a sobreposição da execução de múltiplas instruções ao longo de estágios distintos do caminho de dados. O processador RISC-V, desenvolvido como padrão aberto pela Universidade da Califórnia em Berkeley, constitui uma plataforma didática e industrial de grande relevância, sendo adotado neste trabalho como objeto de estudo.

Este relatório documenta o desenvolvimento do Trabalho Prático 2 da disciplina de Arquitetura de Computadores III, cujo objetivo central é a implementação, em linguagem de descrição de hardware Verilog, de um processador RISC-V com pipeline de 5 estágios: **IF** (Instruction Fetch), **ID** (Instruction Decode), **EX** (Execute), **MEM** (Memory Access) e **WB** (Write Back).

O desenvolvimento foi conduzido em etapas progressivas, cada uma acrescentando complexidade ao caminho de dados base:

1. Implementação da unidade de desvio condicional (*BranchUnit*) e controle de fluxo;
2. Execução de programa de teste com inserção estática de `NOP`s para resolução de *hazards*;
3. Implementação em hardware das unidades de *Forwarding* e de Detecção de *Hazard*.

Ao longo do relatório são apresentadas análises quantitativas de desempenho incluindo contagem de *stalls*, *flushes* e cálculo do CPI bem como explicações conceituais sobre os fenômenos observados. O objetivo final é consolidar a compreensão prática dos mecanismos que limitam e otimizam o desempenho de processadores pipelinados.

2 Descrição do Desenvolvimento

2.1 Arquitetura Implementada

O processador foi implementado em Verilog RTL seguindo o modelo clássico de pipeline de 5 estágios. Os registradores de pipeline (IF/ID, ID/EX, EX/MEM e MEM/WB) armazenam os sinais de controle e dados necessários para a propagação correta das instruções entre estágios.

Os módulos principais são:

- **RISCVCPU**: módulo *top-level*, integra todos os estágios;
- **InstructionMemory** e **DataMemory**: memórias de instrução e dados;
- **RegisterFile**: banco de 32 registradores de 32 bits;
- **ALU**: unidade aritmético-lógica com suporte às operações RISC-V;
- **BranchUnit**: calcula o endereço-alvo e a condição de desvio;
- **ForwardingUnit**: implementa *forwarding* MEM→EX e WB→EX;
- **HazardDetectionUnit**: detecta dependências *load-use* e emite *stalls*;
- **PipelineStats**: contador de ciclos, instruções, *stalls* e *flushes* de *branch*.

2.2 Decisões de Projeto

Resolução de *branch* no estágio EX: Optou-se por calcular a condição de desvio e o endereço-alvo no estágio EX (e não ID), simplificando o caminho de dados às custas de uma penalidade fixa de 2 ciclos por *branch* tomado (os registradores IF/ID e ID/EX são purgados com `NOP`s).

Prioridade de *forwarding*: A `ForwardingUnit` foi projetada com prioridade explícita para o estágio MEM sobre o estágio WB. Isso garante que, em cadeias de dependências rápidas, o valor mais recente (produzido pelo estágio mais avançado no pipeline) seja sempre utilizado pela ALU.

Detecção de *load-use* com 1 stall: A `HazardDetectionUnit` compara o registrador de destino da instrução *load* presente no estágio ID/EX com os operandos lidos pela instrução seguinte (no estágio IF/ID). Ao detectar a dependência, congela o PC e o registrador IF/ID por um ciclo, ao mesmo tempo em que insere um `NOP` forçado no registrador ID/EX.

3 Resultados Obtidos

3.1 Parte 1 Branch e Controle de Fluxo

3.1.1 Descrição

A `BranchUnit` foi implementada para suportar a instrução `beq`. O cálculo do endereço de destino (`branch_target`) e a avaliação da condição (`rs1 == rs2`) ocorrem no estágio EX. Quando o desvio é tomado, a *flag* `branch_taken` aciona o *flush* dos registradores IF/ID e ID/EX, substituindo as instruções já buscadas por `NOP`s e atualizando o PC.

3.1.2 Análise de Desempenho Parte 1

Nesta etapa, o programa de teste executado apresentou dependências de dados e um desvio condicional tomado. Como o hardware de tratamento de *hazards* de dados ainda não estava presente, as dependências resultaram em cálculos incorretos (falhas de validação), mas permitiram registrar a linha de base de ciclos e instruções para o fluxo basal do pipeline.

Tabela 1: Contadores ao final da Parte 1

Métrica	**Valor**
Ciclos totais	14
Instruções completadas	7
Stalls (load-use)	0
Flushes de *branch*	1
CPI	2.00

Por que *stalls* ocorrem (Parte 1): Nesta etapa ainda não havia hardware de detecção de hazard; *stalls* de dados seriam gerados por dependências RAW não resolvidas. Como o programa de teste da Parte 1 não possuía tais dependências, nenhum *stall* foi registrado.

Quando *forwarding* resolve dependências (Parte 1): A `ForwardingUnit` ainda não estava implementada. Consequentemente, qualquer dependência RAW de 1 ou 2 ciclos resultaria em leitura de valor incorreto sem tratamento explícito.

Impacto do *branch* no desempenho (Parte 1): Como há um branch tomado no programa de teste, o CPI ainda está acima do ideal devido à penalidade de branch e ausência de otimizações de dados.

3.2 Parte 2 Execução com NOPs Estáticos

3.2.1 Descrição

Antes da inserção dos controles de *hazard* em hardware, o programa original apresentava dependências de dados intransponíveis. Para solucionar este problema estaticamente, o código *assembly* configurado na *task* `load_program_full_dependencies` no *testbench* (`tb_RISCVCPU.v`) foi modificado com inserção manual de `NOP`s após instruções de *load* e aritméticas dependentes. Os *offsets* dos *labels* de *branch* foram recalculados conforme os bytes extras inseridos.

3.2.2 Análise de Desempenho Parte 2

A inserção de `NOP`s estáticos resolveu as dependências, mas ao custo de aumentar artificialmente o total de ciclos e o CPI, pois instruções inúteis ocupam os estágios do pipeline.

Tabela 2: Contadores ao final da Parte 2 (NOPs estáticos)

Métrica	**Valor**
Ciclos totais	*22*
Instruções completadas	*7*
Stalls de hardware	0
Flushes de *branch*	*1*
CPI	*3.14*

Por que *stalls* ocorrem (Parte 2): Os *stalls* desta parte são **estáticos** não emitidos pelo hardware, mas representados pelos `NOP`s inseridos pelo programador. Cada `NOP` consome um ciclo de pipeline sem executar trabalho útil, degradando o CPI. A causa raiz são as dependências *Read-After-Write* (RAW): uma instrução tenta ler um registrador cujo valor ainda não foi escrito pela instrução anterior no estágio WB.

Quando *forwarding* resolve dependências (Parte 2): Ainda sem *forwarding*, dependências de 1 ou 2 ciclos exigem 1 ou 2 `NOP`s respectivamente. Com *forwarding* (Parte 3), essas dependências seriam resolvidas sem nenhum `NOP`, pois o resultado pode ser repassado diretamente dos estágios MEM ou WB para a entrada da ALU no estágio EX.

Impacto do *branch* no desempenho (Parte 2): Cada *branch* tomado gera *flush* de 2 instruções do pipeline (penalidade de 2 ciclos). Com vários *branches* no programa de teste, a soma dessas penalidades contribui significativamente para a elevação do CPI em relação ao ideal de 1,0.

3.3 Parte 3 Hazard Detection e Forwarding em Hardware

3.3.1 Descrição

Para eliminar a necessidade de `NOP`s estáticos e tratar automaticamente as dependências, o hardware do processador foi expandido com dois módulos:

- **ForwardingUnit:** Repassa resultados dos estágios MEM e WB diretamente para as entradas da ALU (estágio EX). A prioridade é MEM > WB, garantindo uso do valor mais recente em cadeias de dependências consecutivas.
- **HazardDetectionUnit:** Detecta dependências *load-use* intransponíveis ao *forwarding* (pois o dado de `lw` só está disponível ao final do estágio MEM). Ao detectar essa condição, congela PC e IF/ID por 1 ciclo e insere um `NOP` em ID/EX (*stall* de hardware).

3.3.2 Análise de Desempenho Parte 3

Tabela 3: Contadores ao final da Parte 3 (hardware completo)

Métrica	**Valor**
Ciclos totais	15
Instruções completadas	7
Stalls de *load-use*	1
Bypasses	6
Flushes de *branch*	1
CPI	2.14

Por que *stalls* ocorrem (Parte 3): Mesmo com *forwarding*, as dependências *load-use* não podem ser completamente eliminadas. Uma instrução `lw` escreve o dado no registrador de destino apenas ao final do estágio MEM. Se a instrução imediatamente seguinte necessitar desse dado como operando na ALU (estágio EX), o dado simplesmente não estará disponível a tempo nem mesmo via *forwarding*. A `HazardDetectionUnit` detecta essa janela e insere exatamente 1 stall (*stall*), suficiente para que o dado esteja pronto.

Quando *forwarding* resolve dependências (Parte 3): O *forwarding* resolve dependências RAW de distância 1 (MEM→EX) e distância 2 (WB→EX) para instruções aritméticas e lógicas, eliminando qualquer necessidade de *stall* nesses casos. A única exceção é o *load-use* de distância 1: como o dado de memória só chega ao final do estágio MEM, ele não pode ser repassado a tempo para o estágio EX da instrução seguinte, exigindo 1 *stall* adicional.

Impacto do *branch* no desempenho (Parte 3): Com a arquitetura atual, cada *branch* tomado resulta em *flush* de 2 instruções (penalidade fixa de 2 ciclos por desvio tomado). Esse custo reflete a decisão de resolver o *branch* apenas no estágio EX. Processadores modernos mitigam esse impacto com *branch predictors* estáticos ou dinâmicos, que especulam sobre o resultado do desvio e cancelam instruções somente quando a predição é errada, reduzindo a penalidade média significativamente.

3.4 Comparativo de Desempenho entre as Etapas

Tabela 4: Comparativo de métricas de desempenho entre as três partes

Métrica	**Parte 1**	**Parte 2**	**Parte 3**
Ciclos totais	14	22	15
Instruções completadas	7	7	7
Stalls (hardware)	0	0	1
NOPs estáticos	0	8	0
Flushes de branch	1	1	1
CPI	2.00	3.14	2.14

A melhoria de desempenho da Parte 2 para a Parte 3 é verificada pela redução no CPI e no número total de ciclos: a remoção dos `NOP`s estáticos (substituídos por *forwarding* e 1 stall de *load-use*) reduz o volume de instruções executadas sem trabalho útil, aumentando a eficiência do pipeline.

3.5 Validação Funcional

A simulação foi concluída com sucesso em todas as partes. Durante a execução, as validações implementadas na *task* `check_expected` do *testbench* confirmaram a integridade dos dados finais:

- Valores corretos nos registradores `x1` a `x7`;
- Valores corretos nas posições monitoradas da `DMemory`;
- Nenhuma falha de asserção reportada pelo simulador.

4 Conclusão

O Trabalho Prático 2 permitiu consolidar, de forma prática e mensurável, os conceitos fundamentais do pipeline de processadores RISC-V. A implementação progressiva partindo do controle de *branch*, passando pela solução estática de *hazards* e chegando ao hardware completo de detecção e *forwarding* evidenciou claramente os *trade-offs* entre complexidade de hardware e eficiência de execução.

Os resultados quantitativos confirmaram que a `ForwardingUnit` elimina a grande maioria das penalidades por dependências RAW, enquanto a `HazardDetectionUnit` trata de forma precisa e cirúrgica os casos restantes de *load-use*. O único custo estrutural inevitável permanece nas penalidades de *branch*, inerentes à decisão de resolução no estágio EX.

A experiência reforça o entendimento de que o desempenho de processadores pipelinados é determinado não apenas pela frequência de clock, mas pelo grau em que o pipeline consegue manter-se cheio de instruções úteis medido diretamente pelo CPI observado.

5 Uso de Inteligência Artificial (IA)

Em conformidade com a Seção 10 do edital, esta seção descreve de forma transparente o uso de ferramentas de Inteligência Artificial generativa ao longo do projeto, seguindo a abordagem **Spec-Driven Development (SDD)**.

Ferramentas Utilizadas

- **Claude:** Utilizado para geração e revisão do relatório LaTeX e criação de issues no Github.
- **Gemini:** Utilizado para geração e revisão do código

Prompts e Contribuições

1. **Prompt 1:** *"Utilize o GitHub CLI e cadastre issues utilizando o comando gh para facilitar a divisão de tarefas do grupo."*

   Uso da IA: Apoio na gestão de projeto e divisão de tarefas.

2. **Prompt 2:** *"Verifique se a alteração que fiz no arquivo Branchunit.v atende aos requisitos estabelecidos no enunciado do projeto."*

   Uso da IA: Validação de código.

3. **Prompt 3:** *"Estou trabalhando em um pipeline RISC-V de 5 estágios que ainda não possui Forwarding Unit e Hazard Detection Unit e preciso garantir que meu código assembly resolve manualmente as dependências RAW via inserção de nops."*

```
task load_program_full_dependencies;
    begin
        cpu.DMemory[0] = 32'd10;

        cpu.IMemory[0] = 32'h00002083;  // lw   x1, 0(x0)      (x1 = mem[0])
        cpu.IMemory[1] = 32'h0000_0013; // nop
        cpu.IMemory[2] = 32'h0000_0013; // nop

        cpu.IMemory[3] = 32'h00508113;  // addi x2, x1, 5      (x2 = x1 + 5)
        cpu.IMemory[4]  = 32'h00000013; // nop

        cpu.IMemory[5] = 32'h00110193; // addi x3, x2, 1       (x3 = x2 + 1)
        cpu.IMemory[6]  = 32'h00000013; // nop

        cpu.IMemory[7] = 32'h00302223; // sw   x3, 4(x0)       (mem[1] = x3)
        cpu.IMemory[8] = 32'h00a18213; // addi x4, x3, 10      (x4 = x3 + 10)
        cpu.IMemory[9]  = 32'h00000013; // nop

        // PC do beq = 40, label esta em PC = 52
        // offset = 52 - 40 = 12 bytes
        cpu.IMemory[10] = 32'h00420663; // beq  x4, x4, label   (sempre tomado)

        cpu.IMemory[11] = 32'h06300293; // addi x5, x0, 99      (deve ser flushado)
        cpu.IMemory[12] = 32'h05800313; // addi x6, x0, 88      (pode ser flushado)

        // label:
        cpu.IMemory[13] = 32'h00120393; // addi x7, x4, 1       (x7 = resultado final)

        cpu.IMemory[14] = 32'h0000000b; // halt                 (Instrucao para finalizar a simulacao)
    end
endtask
```

Uso da IA: Geração de código/revisão.

4. **Prompt 4:** *"Preciso de ajuda para implementar a lógica de detecção de hazard do tipo load-use no módulo HazardDetectionUnit de um processador RISC-V escrito em Verilog. O problema que temos é que o forwarding padrão não resolve dependências de load-use sozinho, já que o dado só sai da memória no estágio MEM, sendo obrigatório inserir um stall de 1 ciclo no pipeline. Para isso, o requisito funcional é que a variável de saída stall deve receber o valor 1 apenas quando a instrução no registrador EX/MEM for um Load, identificada por exmem_op igual a LW, o registrador de destino desse load não for o zero, e esse mesmo registrador destino for igual a um dos registradores fonte da instrução atual em ID/EX, ou seja, igual a idex_rs1 ou idex_rs2. Como restrições de implementação, o código que você gerar deve ser feito obrigatoriamente para caber dentro do bloco always que já existe no arquivo, sem criar ou alterar as portas de entrada e saída do módulo original, e considerando que a constante LW já foi declarada no topo do arquivo como 7'b000_0011."*

   Uso da IA: Geração de código Verilog.

5. **Prompt 5:** *"Eu tenho que resolver o problema de hazard de dados e isso está acontecendo porque durante a execução que é feita usando o pipeline (busca a instrução na memória, vê o comando, executa, lê ou escreve na memória e salva no registrador) quando eu tenho mais de um código que o resultado de um depende do resultado do outro, por exemplo* `sub x1, x2, x3, add x4, x1, x5`*, o valor de* `x1` *no* `add` *estaria errado. Eu pensei que para resolver esse problema, se possível, teria como ao invés de esperar a resposta ser salva no registrador, eu pegar o valor que foi lido ou escrito na memória e enviá-lo para a próxima linha de código que está sendo executada e com o valor já atualizado (nesse caso o* `sub` *faz* `x1 = x2(3) - x3(1) -> x1 = 2`*, e eu já passo esse valor para o* `add`*, que antes o* `x1=0` *mas passo o* `x1=2`*). Teria como fazer isso?"*

   Uso da IA: Tirar dúvidas conceituais.

6. **Prompt 6:** *"Esse código em Verilog está certo, para resolver o problema anterior? Esse é o código:"*

```
module ForwardingUnit (
    input  [4:0] idex_rs1,
    input  [4:0] idex_rs2,
    input  [4:0] exmem_rd,
    input  [4:0] memwb_rd,
    input  [6:0] exmem_op,
    input  [6:0] memwb_op,
    output reg [1:0] forwardA,
    output reg [1:0] forwardB
);
    localparam NO_FORWARD  = 2'b00;
    localparam FROM_MEM    = 2'b01;
    localparam FROM_WB_ALU = 2'b10;
    localparam FROM_WB_LD  = 2'b11;
    localparam LW    = 7'b000_0011;
    localparam ALUop = 7'b001_0011;
    initial begin
      forwardA = NO_FORWARD;
      forwardB = NO_FORWARD;
    end
    always @(*) begin
        forwardA = NO_FORWARD;
        forwardB = NO_FORWARD;
        // Operando A (rs1)
        // 3a prioridade: WB via Load
        if (memwb_rd == idex_rs1 && memwb_rd != 5'b0 && memwb_op == LW)
            forwardA = FROM_WB_LD;
        // 2a prioridade: WB via ALU
        if (memwb_rd == idex_rs1 && memwb_rd != 5'b0 && memwb_op == ALUop)
            forwardA = FROM_WB_ALU;
        // 1a prioridade: MEM (sobrescreve tudo, valor mais recente)
        if (exmem_rd == idex_rs1 && exmem_rd != 5'b0 &&
            (exmem_op == ALUop || exmem_op == LW))
            forwardA = FROM_MEM;
        // Operando B (rs2)
        // 3a prioridade: WB via Load
        if (memwb_rd == idex_rs2 && memwb_rd != 5'b0 && memwb_op == LW)
            forwardB = FROM_WB_LD;
        // 2a prioridade: WB via ALU
        if (memwb_rd == idex_rs2 && memwb_rd != 5'b0 && memwb_op == ALUop)
            forwardB = FROM_WB_ALU;
        // 1a prioridade: MEM (sobrescreve tudo, valor mais recente)
        if (exmem_rd == idex_rs2 && exmem_rd != 5'b0 &&
            (exmem_op == ALUop || exmem_op == LW))
            forwardB = FROM_MEM;
    end
endmodule
```

Uso da IA: Validação de código.

7. **Prompt 7:** *"Precisamos alterar o nosso arquivo.tex para ficar de acordo com o enunciado do trabalho, incluindo análise de CPI por etapa, por que stalls ocorrem, quando forwarding resolve, impacto do branch, e seções obrigatórias de introdução, desenvolvimento, resultados, conclusão e uso de IA]."*
Uso da IA: Apoio na escrita e reestruturação do relatório.

6 Repositório no GitHub

Link: https://github.com/matheusmra/TP02-ARQ3.